

Image-Based Visual Servoing with Model Predictive Control for Color-Guided Box Delivery on a Differential-Drive Mobile Robot

Josue Ureña Valencia, César Arellano Arellano, Jose Eduardo Sanchez Martinez, and Rafael André Gamiz Salazar

Tecnológico de Monterrey, Monterrey, Nuevo León, México
{A01738940, A00839373, A01738476, A00838280}@tec.mx

Abstract

Este trabajo presenta un sistema de visual servoing basado en imagen para la recolección guiada por color de cajas mediante un robot móvil diferencial. El robot identifica cajas verdes, rosas y amarillas utilizando visión clásica, específicamente segmentación en espacio HSV, operaciones morfológicas y análisis de contornos. A partir de la detección visual, se estiman dos variables principales: la distancia aproximada a la caja, ρ , y el error angular respecto al centro de la imagen, α . Estas variables conforman el estado visual utilizado por un controlador Sampled Model Predictive Control, encargado de generar comandos de velocidad lineal y angular para aproximar al robot al objetivo de manera estable y respetando restricciones físicas. El sistema también incorpora una máquina de estados finitos que coordina la selección del color, el seguimiento visual, la recolección simulada y la navegación posterior por odometría hacia la estación de entrega o hacia waypoints definidos por el operador. Los resultados experimentales muestran que el robot puede detectar el objetivo, aproximarse a él en lazo cerrado con la cámara y completar la secuencia de operación en tiempo real sobre una plataforma física.

1 Introducción

El visual servoing es una técnica de control que utiliza retroalimentación visual para regular el movimiento de un sistema hacia una configuración deseada [1]. A diferencia de los esquemas basados en trayectorias planificadas previamente, permite al robot reaccionar en tiempo real a la información percibida por su sistema de visión, lo cual es mejor en entornos dinámicos o parcialmente desconocidos, como en las aplicaciones industriales de manipulación y logística.

Las técnicas de visual servoing se clasifican comúnmente en enfoques basados en imagen (IBVS: image-based visual servoing) y basados en pose (PBVS: pose-based visual servoing) [2]. En este trabajo se adopta un enfoque IBVS, en el cual el control se formula directamente sobre características extraídas de la imagen, sin requerir reconstrucción tridimensional ni calibración intrínseca precisa de la cámara. Esta decisión se justifica con mayor detalle en la Sección 2.3.

Los controladores clásicos de IBVS se basan típicamente en leyes proporcionales derivadas de la matriz de interacción (image Jacobian) [1]. Si bien son simples y efectivos en condiciones ideales, no manejan de forma natural las restricciones físicas del actuador, tales como saturaciones de velocidad o aceleración, ni permiten anticipar el comportamiento futuro del sistema. Estas limitaciones son relevantes en robots móviles diferenciales, cuya cinemática no holónoma restringe los movimientos instantáneamente posibles. En este contexto, el control predictivo basado en modelo (MPC: Model Predictive Control) ha emergido como una alternativa atractiva, puesto que, formula el problema de control como una optimización sobre un horizonte de predicción finito, en la cual se minimiza una función de costo sujeta a restricciones explícitas

de estado y de entrada. Trabajos previos han demostrado la viabilidad de combinar MPC con visual servoing tanto para seguimiento de trayectorias visuales sobre robots móviles con ruedas [3] como para guiado visual en plataformas no holónomas con restricciones [4], confirmando su aplicabilidad para plataformas con dinámicas restringidas similares a la utilizada en este trabajo.

En este trabajo se presenta un sistema completo de visual servoing formulado como un problema de control óptimo, implementado sobre un robot móvil diferencial, el Puzzlebot. El sistema resuelve una tarea de entrega autónoma de cajas guiada por color. Un operador especifica un color objetivo mediante una pulsación de tecla, y el robot identifica autónomamente la caja correspondiente, se aproxima a ella mediante un controlador MPC en lazo cerrado con la cámara, y regresa a la estación de entrega para aceptar nuevas órdenes. La percepción se basa exclusivamente en visión clásica, evitando el uso de redes neuronales o métodos de aprendizaje profundo, dada la simplicidad y predictibilidad de los métodos clásicos en entornos controlados y su viabilidad de ejecución en tiempo real sobre hardware.

Las contribuciones principales del trabajo son las siguientes:

- Un pipeline de percepción basado en visión clásica capaz de detectar y discriminar cajas de tres colores: verde, rosa y amarillo mediante segmentación en espacio HSV, operaciones morfológicas y análisis de contornos.
- Una representación de estado en coordenadas polares relativas (ρ, α) , donde ρ es la distancia estimada a la caja objetivo obtenida a partir del modelo de proyección pinhole de la cámara, y α es el ángulo de orientación relativa.
- Un controlador Sampled Model Predictive Control con modelo de predicción calibrado empíricamente sobre la plataforma real, que respeta las restricciones de velocidad lineal y angular del robot diferencial y penaliza la suavidad del control.
- Una máquina de estados finitos que coordina las distintas fases de la misión: espera de comando, seguimiento visual, recolección simulada y navegación posterior por odometría hacia la estación de entrega o hacia waypoints especificados por el operador.

2 Definición del Problema

2.1 Descripción de la Tarea

El sistema propuesto resuelve una tarea de entrega autónoma de cajas en un entorno industrial simplificado. La escena consta de 6 cajas distribuidas de forma dispersa, todas dentro del campo de visión del robot desde la estación de entrega. Las cajas están coloreadas en tres colores distintos: verde, rosa y amarillo. El Puzzlebot parte desde la estación de entrega, definida como su pose inicial $P_0 = (0, 0, 0)$ en el marco de referencia de la odometría.

Un operador indica al robot qué caja recoger mediante una pulsación de tecla durante la ejecución: la tecla **g** solicita una caja verde, la tecla **p** una caja rosa y la tecla **y** una caja amarilla. El operador le indica a donde enviar la caja mediante un waypoint $P_w = (x, y, \theta)$.

Cuando existen varias cajas del color solicitado dentro del campo de visión, el sistema selecciona la caja más cercana al robot. Este criterio se aproxima mediante el área aparente del contorno detectado en la imagen, por ejemplo, a mayor área aparente, menor distancia estimada. Una vez identificada la caja objetivo, el robot inicia una fase de aproximación en lazo cerrado con la cámara, en la cual un controlador óptimo regula simultáneamente la distancia a la caja y la orientación del robot hacia ella.

Cuando el robot alcanza una distancia objetivo ρ^* predefinida, se considera que la caja ha sido recogida (la operación de recolección se simula mediante el alcance visual del objetivo). En ese momento el sistema transita a un estado de espera en el cual el operador puede decidir el siguiente destino del robot mediante dos comandos disponibles: la tecla **h** ordena el regreso home $P_0 = (0, 0, 0)$, mientras que el comando **w x y** ordena al robot navegar hacia un waypoint con coordenadas (x, y) especificadas por el operador.

2.2 Supuestos del Sistema

El diseño del sistema se basa en una serie de supuestos que delimitan su alcance y establecen las condiciones bajo las cuales se garantiza su correcto funcionamiento 4.4.

- **Visibilidad inicial:** Todas las cajas objetivo son visibles desde la estación de entrega. El robot no requiere ejecutar una fase de búsqueda exploratoria para localizar las cajas.
- **Existencia del color objetivo:** El color solicitado por el operador corresponde siempre a al menos una caja presente en la escena. El manejo de comandos inválidos o de cajas inexistentes no se encuentra dentro del alcance del proyecto.
- **Separación de cajas del mismo color:** Las cajas del mismo color se asumen suficientemente separadas para que la segmentación produzca contornos individuales. Si dos cajas del mismo color están adyacentes, el pipeline de visión las trata como un único objeto, lo cual constituye una limitación documentada.
- **Iluminación:** Las condiciones de iluminación durante la operación se asumen razonablemente constantes y suficientes para garantizar una segmentación HSV confiable de los tres colores objetivo.
- **Modelo cinemático:** El robot se modela como un vehículo diferencial tipo unicycle, asumiendo rodadura pura sin deslizamiento significativo de las ruedas. Esta hipótesis es razonable para superficies planas y velocidades moderadas.
- **Dimensiones conocidas de la caja:** El ancho real de las cajas se asume conocido y uniforme. Este parámetro es necesario para estimar la distancia a la caja a partir del modelo de proyección pinhole de la cámara, como se describe en la Sección 3.4.3.
- **Comandos en estado de espera:** El sistema acepta comandos del operador únicamente cuando se encuentra en el estado `WAIT_COMMAND`. Las pulsaciones de tecla durante fases activas de aproximación o regreso son ignoradas, lo cual evita transiciones inconsistentes en la máquina de estados.
- **Odometría como referencia para el regreso:** Tanto la fase de regreso a la estación de entrega como la navegación hacia waypoints arbitrarios dependen exclusivamente de la estimación de pose por odometría (dead reckoning), sin retroalimentación visual ni corrección por marcadores externos. Se reconoce que el error de odometría se acumula a lo largo de la misión, lo cual constituye una limitación del sistema y afecta la precisión de la navegación a medida que se encadenan múltiples acciones.

2.3 Justificación del Enfoque IBVS

La elección de un esquema basado en imagen (IBVS) sobre uno basado en pose (PBVS) responde a las características particulares de nuestra tarea y de la plataforma utilizada. Primero, el sistema debe operar en entornos no calibrados: no se dispone de un mapa previo del espacio de trabajo, no se utilizan coordenadas mundo predefinidas, y la disposición física de las cajas puede variar entre cada run. Por tanto, el robot debe ser capaz de ejecutar la tarea en cualquier configuración donde las cajas sean visibles desde la posición inicial, lo cual descarta enfoques basados en *waypoints* o trayectorias planificadas.

En segundo lugar, un enfoque PBVS clásico requeriría una calibración intrínseca precisa de la cámara, un modelo geométrico tridimensional detallado de cada caja, y una etapa de estimación de pose mediante correspondencia entre puntos 2D-3D. Estos requerimientos incrementan mucho la complejidad del sistema y su sensibilidad ante errores de modelado, sin aportar beneficios sustanciales para una tarea cuya información relevante puede extraerse directamente del plano de imagen.

En la formulación propuesta, las variables que entran al controlador no son píxeles, sino dos cantidades derivadas con significado físico claro: la distancia estimada ρ (en metros) y el ángulo relativo α (en radianes). Ambas se obtienen a partir de mediciones realizadas en el plano de imagen y del modelo de proyección pinhole de la cámara, conservando así el espíritu de IBVS al evitar una reconstrucción tridimensional completa. Esta representación combina la robustez del IBVS frente a errores de calibración con la interpretabilidad de un marco polar local, facilitando tanto el diseño del controlador óptimo como el análisis cualitativo de su comportamiento.

3 Materiales y Métodos

Esta sección describe los componentes físicos y lógicos que conforman el sistema implementado, así como la formulación matemática que sustenta el pipeline completo de visual servoing. Se presenta primero la plataforma de hardware utilizada, seguida de las herramientas de software, el pipeline de percepción basado en visión clásica, el modelado del sistema, la formulación del controlador óptimo, el controlador de regreso por odometría, la máquina de estados finitos que coordina las fases de operación, y finalmente la integración de todos los componentes en un pipeline unificado.

3.1 Hardware

La plataforma física utilizada consiste en un robot móvil diferencial Puzzlebot [7], una cámara monocular RGB y una plataforma de cómputo embebida que ejecuta el pipeline completo de percepción, control y comunicación. A continuación se describen los componentes principales.

Robot Puzzlebot

El Puzzlebot es una plataforma educativa de Manchester Robotics basada en una arquitectura de tracción diferencial, con dos ruedas motrices independientes y una rueda de apoyo. Sus parámetros geométricos relevantes para el modelo cinemático son el radio de las ruedas $r = 0.05$ m y la distancia entre ruedas $l = 0.185$ m. El robot incorpora encoders en cada rueda motriz, los cuales proporcionan las mediciones angulares necesarias para la estimación de pose por odometría descrita en la Sección 3.6. La electrónica de a bordo expone los comandos de velocidad lineal y angular a través de tópicos estándar de ROS 2, lo cual facilita la integración con el resto del pipeline.

Los límites de velocidad utilizados durante la operación del sistema fueron establecidos en $v_{\max} = 0.12$ m/s y $\omega_{\max} = 0.30$ rad/s para la fase de aproximación visual, y en $v_{\max} = 0.10$ m/s y $\omega_{\max} = 0.60$ rad/s para la fase de regreso por odometría. Estos límites son consistentes con la dinámica nominal del Puzzlebot y permiten un funcionamiento seguro durante las pruebas experimentales.

Sistema de visión

La captura de imágenes se realiza mediante una Raspberry Pi Camera V2 montada en la parte frontal del robot, con su eje óptico aproximadamente alineado con el eje de avance del Puzzlebot. La cámara publica imágenes en formato BGR a través del tópico ROS `/video_source/raw`, y estas son procesadas en tiempo real por el nodo de percepción.

El sistema utiliza una distancia focal efectiva de $f = 1300$ px, estimada empíricamente sobre la plataforma real. Este valor se emplea en el modelo de proyección pinhole para inferir la distancia a la caja objetivo a partir del ancho aparente del contorno detectado, como se detalla en la Sección 3.4.3.

Objetos de la escena

La escena contiene 6 cajas idénticas en geometría pero diferenciadas por color: cajas verdes, rosas y amarillas. El ancho real de cada caja es de $W_{\text{caja}} = 0.06$ m, parámetro conocido y utilizado por el sistema para la estimación de distancia. Las cajas se disponen de forma dispersa sobre el plano de operación, pero todas dentro del campo de visión del robot desde la estación de entrega.

Plataforma de cómputo

El pipeline completo ejecuta una distribución Ubuntu compatible con ROS 2. La comunicación entre el nodo de control y el robot se realiza exclusivamente mediante tópicos ROS, lo cual garantiza una arquitectura modular y desacoplada.

La Tabla 1 resume los parámetros físicos del Puzzlebot, la cámara y las cajas utilizados en el modelado del sistema. Los parámetros del robot (r , l) corresponden a las especificaciones del fabricante, mientras que los límites de velocidad fueron ajustados experimentalmente para garantizar una operación estable. La distancia focal f y el ancho de la caja W_{caja} fueron determinados de forma empírica sobre la plataforma real.

Table 1: Parámetros físicos del sistema.

Parámetro	Símbolo	Valor
Radio de las ruedas	r	0.05 m
Distancia entre ruedas	l	0.185 m
Velocidad lineal máxima (MPC)	$v_{\max}$	0.12 m/s
Velocidad angular máxima (MPC)	$\omega_{\max}$	0.30 rad/s
Velocidad lineal máxima (regreso)	$v_{\max}^{\text{ret}}$	0.10 m/s
Velocidad angular máxima (regreso)	$\omega_{\max}^{\text{ret}}$	0.60 rad/s
Distancia focal efectiva	f	1300 px
Ancho real de la caja	W_{caja}	0.06 m
Distancia objetivo de aproximación	ρ^*	0.10 m

3.2 Software

La arquitectura de software del sistema se basa en ROS 2, que proporciona un marco de comunicación distribuida mediante tópicos, servicios y nodos.

El pipeline completo está implementado en Python y se organiza en tres nodos ROS 2 independientes:

- **Nodo de odometría** (`puzzlebot_odometry`). Recibe las velocidades angulares medidas en cada rueda a través de los tópicos `/VelocityEncR` y `/VelocityEncL`, e integra el modelo cinemático del robot diferencial para estimar la pose actual (x, y, θ) en el marco de referencia inicial. La pose estimada se publica a una frecuencia de 100 Hz en el tópico `/odom` mediante el mensaje estándar `nav_msgs/Odometry`.
- **Nodo de teleoperación** (`teleop_color`). Captura las pulsaciones de tecla del operador desde la terminal mediante el módulo `termios` y publica diversos mensajes de control en el tópico `/box_color_command`. Los comandos soportados incluyen: selección de color objetivo (`g`, `p`, y para verde, rosa y amarillo respectivamente), regreso a la estación de entrega (`h`), navegación a un waypoint con coordenadas especificadas (`w x y`), cancelación de la misión actual (`c`) y terminación del nodo (`q`). La separación de este nodo del nodo principal permite controlar el sistema desde cualquier terminal con acceso al *rosgraph*, sin necesidad de interactuar directamente con la ventana de visualización del pipeline de percepción.
- **Nodo de percepción y control** (`puzzlebot_box_mpc_node`). Suscribe al tópico de imagen `/video_source/raw`, al tópico de odometría `/odom` y al tópico de comandos de color `/box_color_command`. Ejecuta el pipeline de visión clásica, el controlador MPC durante la fase de aproximación, el controlador proporcional durante la fase de regreso, y la máquina de estados que coordina ambas fases. Publica los comandos de velocidad lineal y angular en el tópico `/cmd_vel` mediante el mensaje `geometry_msgs/Twist`.

Adicionalmente, durante la fase de desarrollo se utilizó un nodo de calibración (`calibration_node`) para identificar empíricamente los parámetros K_ρ y K_α del modelo de predicción del MPC, descritos en la Sección 3.4.4. Este nodo no forma parte del pipeline operativo final, pero se incluye en el repositorio del proyecto para permitir la recalibración del sistema en condiciones distintas a las utilizadas durante este trabajo.

El procesamiento de imágenes se realiza mediante la biblioteca OpenCV [6], que proporciona implementaciones eficientes y validadas de operaciones de bajo nivel tales como conversión entre espacios de color, segmentación por umbralizado, detección de contornos y cálculo de momentos de imagen. El uso de OpenCV permite mantener tiempos de procesamiento compatibles con la operación en tiempo real sobre el hardware disponible para nuestro proyecto.

Para las operaciones numéricas auxiliares se emplea NumPy, mientras que la conversión entre representaciones de orientación (cuaterniones y ángulos de Euler) requerida por la odometría se realiza mediante la biblioteca `transforms3d`. La interacción con el operador se implementa en el nodo de teleoperación mediante el módulo `termios` de la biblioteca estándar de Python, que permite la lectura no bloqueante de pulsaciones de tecla desde una terminal en modo *raw*.

3.3 Pipeline de Percepción Clásica

El pipeline de percepción está diseñado bajo el principio de utilizar exclusivamente técnicas de visión clásica, evitando deliberadamente métodos basados en aprendizaje profundo o redes

neuronales. Esta decisión responde a tres consideraciones principales: la simplicidad y predictibilidad de los métodos clásicos en entornos controlados, la ausencia de requerimientos de entrenamiento o conjuntos de datos anotados y la viabilidad de ejecución en tiempo real sobre hardware más modesto. El pipeline se organiza en tres etapas secuenciales: segmentación por color en espacio HSV, procesamiento morfológico de la máscara binaria y extracción de contornos, y finalmente cálculo de características visuales con selección del objetivo.

3.3.1 Segmentación por Color en Espacio HSV

Cada imagen capturada por la cámara es convertida del espacio de color nativo BGR al espacio HSV (Hue, Saturation, Value) mediante la función `cv2.cvtColor` de OpenCV. La transformación al espacio HSV se justifica por su mayor robustez frente a variaciones de iluminación en comparación con BGR. En HSV, la información cromática del píxel queda concentrada en el canal H (matiz), mientras que la intensidad luminosa se separa en el canal V (valor). Esto permite definir rangos de color que se mantienen aproximadamente constantes ante cambios moderados en la iluminación de la escena, lo que es importante en entornos no controlados donde la luz puede variar a lo largo de la operación, como ocurre en escenarios no completamente controlados.

Para cada uno de los tres colores objetivo se define un par de umbrales ($\mathbf{h}_{lo}, \mathbf{h}_{hi}$) en el espacio HSV, calibrados empíricamente sobre la plataforma real. La Tabla 2 resume los rangos utilizados.

Table 2: Rangos de segmentación HSV utilizados por cada color objetivo.		
Color	Umbral inferior (H, S, V)	Umbral superior (H, S, V)
Verde	(35, 70, 100)	(90, 255, 255)
Rosa	(140, 60, 100)	(175, 255, 255)
Amarillo	(20, 80, 120)	(38, 255, 255)

Una vez definidos los umbrales del color solicitado, se aplica la operación `cv2.inRange`, la cual produce una máscara binaria $\mathbf{M}(u, v) \in \{0, 255\}$ donde cada píxel toma el valor 255 si su tripleta HSV se encuentra dentro del rango definido y 0 en caso contrario.

3.3.2 Procesamiento Morfológico y Extracción de Contornos

La máscara binaria obtenida en la etapa anterior generalmente contiene ruido residual (píxeles aislados clasificados erróneamente) y huecos internos en las regiones detectadas. Para mitigar estos efectos se aplican dos operaciones morfológicas con un elemento estructurante cuadrado de 5×5 píxeles. Primero, una operación de apertura (`MORPH_OPEN`), que consiste en una erosión seguida de una dilatación, elimina componentes pequeños no conectados. Posteriormente, una operación de cierre (`MORPH_CLOSE`), que consiste en una dilatación seguida de una erosión, rellena huecos internos en las regiones detectadas. El resultado es una máscara depurada donde cada componente conexa corresponde, idealmente, a una caja del color objetivo.

Sobre la máscara depurada se ejecuta la función `cv2.findContours` en modo `RETR_EXTERNAL`, que extrae únicamente los contornos externos de cada componente conexa. Para descartar detecciones espurias (reflejos de luz, sombras o pequeñas regiones residuales del fondo) se aplica un filtro de área mínima: solo los contornos con área mayor a $A_{\min} = 800$ píxeles cuadrados son considerados detecciones válidas. Adicionalmente, se descartan los contornos cuyo momento de orden cero M_{00} sea nulo, evitando divisiones por cero en el cálculo del centroide.

3.3.3 Extracción de Características y Selección del Objetivo

Para cada contorno válido se calculan dos descriptores que serán utilizados posteriormente por el módulo de control: el centroide y el bounding box. El centroide (u_c, v_c) se obtiene a partir de los momentos de imagen de orden cero y de orden uno:

$$u_c = \frac{M_{10}}{M_{00}}, \quad v_c = \frac{M_{01}}{M_{00}}, \quad (1)$$

donde M_{ij} representa el momento de imagen de orden (i, j) calculado sobre la región delimitada por el contorno. Adicionalmente, se calcula el bounding box mediante `cv2.boundingRect`, del cual se conserva el ancho aparente w_{px} . Esta cantidad es crítica porque, como se mostrará en la Sección 3.4.3, permite estimar la distancia real a la caja a partir del modelo de proyección pinhole de la cámara.

Cuando existen múltiples detecciones del color solicitado, el sistema selecciona como objetivo aquella con mayor área. Este criterio se fundamenta en una observación geométrica simple: bajo el modelo de proyección perspectiva, el área aparente de un objeto en la imagen es inversamente proporcional al cuadrado de su distancia a la cámara. Por tanto, a mayor área aparente, menor distancia estimada al robot. Aunque este criterio podría fallar si dos cajas del mismo color estuvieran a distancias similares pero orientadas de forma distinta, en el rango de operación nominal del sistema constituye una aproximación adecuada y computacionalmente eficiente. Si no se detecta ninguna caja del color objetivo, el sistema permanece en espera durante un número máximo de frames antes de declarar la pérdida del objetivo, mecanismo descrito en la Sección 3.7.

3.4 Modelado del Sistema

El diseño del controlador predictivo requiere una representación matemática del comportamiento del sistema y de la relación entre la información visual extraída de la imagen y las variables físicas que deben regularse durante la tarea de visual servoing. En esta sección se presenta el modelo utilizado para describir tanto la cinemática del robot diferencial como la representación visual empleada por el controlador.

La formulación adoptada busca mantener la simplicidad y robustez características del enfoque IBVS, evitando la reconstrucción completa de la pose tridimensional del objeto y trabajando directamente sobre variables derivadas del plano de imagen con significado físico claro.

3.4.1 Notación

A lo largo del trabajo se emplea la siguiente notación para describir el estado visual, el estado cinemático y las entradas de control del sistema. El marco de referencia del robot se denota como $\mathcal{F}_R = \{X_R, Y_R\}$, cuyo origen coincide con el centro geométrico del Puzzlebot. La orientación del robot respecto al marco global de odometría se representa mediante el ángulo θ .

La pose del robot en el plano se define como $\mathbf{p} = [x, y, \theta]^T$, donde (x, y) representan la posición cartesiana del robot respecto al marco de odometría y θ su orientación angular.

El estado visual utilizado por el controlador MPC se define como $\mathbf{x}_v = [\rho, \alpha]^T$, donde ρ es la distancia estimada entre el robot y la caja objetivo, y α es el ángulo relativo entre el eje óptico de la cámara y la dirección hacia la caja.

Las entradas de control del sistema se definen como $\mathbf{u} = [v, \omega]^T$, donde v corresponde a la velocidad lineal y ω la velocidad angular del robot.

3.4.2 Modelo Cinemático del Robot Diferencial

El Puzzlebot se modela como un vehículo diferencial tipo uniciclo bajo la hipótesis de rodadura pura sin deslizamiento lateral. El modelo cinemático continuo del robot está dado por

$$\dot{x} = v \cos(\theta) \quad (2)$$

$$\dot{y} = v \sin(\theta) \quad (3)$$

$$\dot{\theta} = \omega \quad (4)$$

donde (x, y, θ) representan la pose del robot y (v, ω) corresponden a las velocidades lineal y angular, respectivamente.

Las velocidades del robot se obtienen a partir de las velocidades tangenciales de las ruedas derecha e izquierda, denotadas como v_r y v_l , respectivamente:

$$v = \frac{v_r + v_l}{2} \quad (5)$$

$$\omega = \frac{v_r - v_l}{l} \quad (6)$$

donde l es la distancia entre ruedas del Puzzlebot.

3.4.3 Estimación de Distancia y Ángulo Relativo

El controlador no opera directamente sobre coordenadas de píxeles, sino sobre una representación polar relativa derivada de la imagen. Las variables visuales utilizadas por el sistema son la distancia estimada a la caja objetivo ρ y el ángulo relativo α .

La estimación de ambas cantidades se realiza utilizando únicamente información geométrica extraída del plano de imagen, evitando la reconstrucción tridimensional completa de la escena y manteniendo así la naturaleza image-based del esquema de visual servoing.

Sea u_c la coordenada horizontal del centroide del objeto detectado y u_0 el centro horizontal de la imagen. El error horizontal en píxeles se define como

$$e_x = u_c - u_0 \quad (7)$$

donde $e_x > 0$ indica que el objeto se encuentra hacia la derecha del centro de la imagen y $e_x < 0$ indica una desviación hacia la izquierda.

Bajo el modelo de proyección pinhole, el ángulo relativo entre el eje óptico de la cámara y la dirección hacia el objeto puede aproximarse mediante

$$\alpha = \tan^{-1} \left(\frac{e_x}{f} \right) \quad (8)$$

donde f es la distancia focal efectiva expresada en píxeles.

La distancia relativa a la caja se estima utilizando el ancho aparente del bounding box detectado en la imagen. Sea W el ancho real conocido de la caja y w_{px} el ancho observado en píxeles. Bajo el modelo de cámara pinhole se obtiene

$$\rho = \frac{fW}{w_{px}} \quad (9)$$

Esta relación refleja el comportamiento geométrico de la proyección perspectiva: conforme el objeto se aproxima a la cámara, su tamaño aparente en la imagen aumenta.

En la implementación experimental, la distancia estimada se filtra mediante un promedio móvil sobre las últimas mediciones disponibles, con el objetivo de reducir fluctuaciones causadas por ruido en la segmentación, pequeñas variaciones del bounding box y vibraciones de la cámara durante el movimiento del robot.

La formulación descrita permite transformar directamente información visual extraída del plano de imagen en variables físicas interpretables por el controlador, evitando la necesidad de estimación explícita de pose tridimensional o calibración geométrica compleja del entorno.

3.4.4 Modelo de Predicción Calibrado

El controlador predictivo utiliza un modelo discreto simplificado en coordenadas polares relativas, formulado directamente sobre las variables visuales utilizadas por el esquema IBVS. El estado del sistema se define como $\mathbf{x}_k = [\rho_k, \alpha_k]^T$, mientras que las entradas de control están dadas por $\mathbf{u}_k = [v_k, \omega_k]^T$. Donde v_k y ω_k representan las velocidades lineal y angular aplicadas al robot en el instante discreto k .

La evolución temporal del sistema se aproxima mediante el modelo discreto:

$$\rho_{k+1} = \rho_k - K_\rho v_k \cos(\alpha_k) \Delta t \quad (10)$$

$$\alpha_{k+1} = \alpha_k - K_\alpha \omega_k \Delta t \quad (11)$$

Donde Δt corresponde al periodo de discretización del controlador y K_ρ , K_α son factores de calibración empírica.

El modelo discreto se obtuvo mediante el desarrollo de un nodo experimental de calibración encargado de ejecutar pruebas controladas sobre el robot real. Durante estas pruebas se aplicaron velocidades lineales y angulares conocidas mientras se registraban continuamente las variables visuales ρ y α , junto con las acciones de control enviadas al robot.

Los datos experimentales obtenidos fueron almacenados en archivos CSV para analizar la relación entre las entradas de control y la evolución real de las variables visuales. A partir de estas mediciones se ajustaron experimentalmente los parámetros K_ρ y K_α , buscando minimizar la discrepancia entre el comportamiento predicho por el modelo y la dinámica observada en la plataforma real.

La inclusión de estos factores calibrados mejora significativamente la correspondencia entre las predicciones utilizadas por el MPC y la respuesta real del sistema, permitiendo un seguimiento visual más estable y consistente durante la aproximación hacia la caja objetivo.

3.5 Formulación del Control Óptimo

La fase de aproximación visual hacia la caja objetivo se formula como un problema de control óptimo en horizonte finito. El objetivo del controlador consiste en generar comandos de velocidad lineal y angular que permitan regular simultáneamente la distancia relativa hacia la caja y el error angular respecto al eje óptico de la cámara, mientras se respetan las restricciones físicas del robot diferencial y se mantiene un comportamiento suave y estable.

3.5.1 Estado y Variables de Control

El problema de control óptimo se formula sobre el estado visual del sistema, definido en términos de las variables relativas utilizadas por el esquema de visual servoing. En cada instante discreto k , el estado del sistema se representa mediante:

$$\mathbf{x}_k = \begin{bmatrix} \rho_k \\ \alpha_k \end{bmatrix}$$

Donde:

- ρ_k representa la distancia estimada entre el robot y la caja objetivo.
- α_k representa el error angular relativo entre el eje óptico de la cámara y la dirección hacia el objetivo visual.

Las entradas de control utilizadas por el MPC se definen como:

$$\mathbf{u}_k = \begin{bmatrix} v_k \\ \omega_k \end{bmatrix}$$

Donde v_k corresponde a la velocidad lineal y ω_k a la velocidad angular aplicada al robot. El objetivo del controlador consiste en regular el estado visual hacia la referencia deseada:

$$\mathbf{x}_{ref} = \begin{bmatrix} \rho^* \\ 0 \end{bmatrix}$$

Donde ρ^* corresponde a la distancia deseada respecto a la caja objetivo. La condición $\alpha = 0$ implica que el objeto se encuentra alineado con el centro de la imagen.

La evolución temporal del estado se predice utilizando el modelo calibrado presentado en la Sección 3.4.4:

$$\rho_{k+1} = \rho_k - K_\rho v_k \cos(\alpha_k) \Delta t$$

$$\alpha_{k+1} = \alpha_k - K_\alpha \omega_k \Delta t$$

Donde Δt es el periodo de discretización del controlador y K_ρ , K_α son factores de calibración obtenidos experimentalmente.

Este modelo predictivo permite estimar la evolución futura de las variables visuales para diferentes secuencias de control, información que posteriormente es utilizada por el MPC para seleccionar la acción óptima en cada iteración.

3.5.2 Función de Costo

El controlador predictivo calcula las acciones de control mediante la minimización de una función de costo definida sobre un horizonte de predicción finito de longitud N . La función objetivo busca reducir simultáneamente el error visual respecto al objetivo, el esfuerzo de control aplicado al robot y las variaciones bruscas entre comandos consecutivos.

La función de costo utilizada se define como:

$$J = \sum_{k=0}^{N-1} \left(q_\rho (\rho_k - \rho^*)^2 + q_\alpha \alpha_k^2 + r_v v_k^2 + r_\omega \omega_k^2 + r_{\Delta v} (v_k - v_{k-1})^2 + r_{\Delta \omega} (\omega_k - \omega_{k-1})^2 \right) \quad (12)$$

Donde:

- q_ρ penaliza el error de distancia respecto a la referencia deseada.
- q_α penaliza el error angular de alineación visual.
- r_v y r_ω penalizan el esfuerzo de control asociado a las velocidades lineal y angular.
- $r_{\Delta v}$ y $r_{\Delta \omega}$ penalizan variaciones bruscas respecto al comando aplicado en la iteración previa del controlador.

La inclusión de términos de suavizado reduce oscilaciones y evita movimientos abruptos del robot, lo cual resulta particularmente importante debido a las limitaciones dinámicas y a la sensibilidad del sistema visual frente a vibraciones durante el movimiento.

En la implementación experimental se utilizaron los siguientes pesos:

$$\begin{aligned}
 q_\rho &= 12.0 \\
 q_\alpha &= 22.0 \\
 r_v &= 1.0 \\
 r_\omega &= 1.2 \\
 r_{\Delta v} &= 2.0 \\
 r_{\Delta \omega} &= 5.0
 \end{aligned}$$

Estos parámetros fueron ajustados experimentalmente buscando un compromiso entre rapidez de convergencia, estabilidad visual y suavidad en las acciones de control.

En particular, se asignó un peso mayor al error angular y a las variaciones de velocidad angular, debido a que oscilaciones excesivas en la orientación del robot pueden provocar pérdida temporal del objetivo visual o movimientos inestables de la cámara.

3.5.3 Restricciones del Sistema

El problema de optimización considera explícitamente restricciones sobre las entradas de control, derivadas de los límites físicos del Puzzlebot y de criterios de seguridad operacional.

Las velocidades lineales y angulares se restringen mediante

$$0 \leq v_k \leq v_{\max} \quad (13)$$

$$-\omega_{\max} \leq \omega_k \leq \omega_{\max} \quad (14)$$

Donde los límites utilizados son:

$$\begin{aligned}
 v_{\max} &= 0.12 \text{ m/s} \\
 \omega_{\max} &= 0.30 \text{ rad/s}
 \end{aligned}$$

Estos límites se seleccionaron considerando las capacidades físicas de la plataforma y buscando evitar movimientos excesivamente rápidos que pudieran comprometer la estabilidad del sistema visual.

Adicionalmente, se incorporaron restricciones de seguridad relacionadas con el alineamiento visual del objetivo. En particular, si el error angular excede un umbral de seguridad, el controlador deshabilita temporalmente el avance lineal del robot y prioriza la corrección de la orientación antes de continuar la aproximación.

En la implementación experimental se utilizaron dos umbrales angulares:

$$|\alpha_k| > 15^\circ \Rightarrow v_k = 0 \quad (15)$$

y

$$|\alpha_k| > 6^\circ \wedge v_{k-1} = 0 \Rightarrow v_k = 0 \quad (16)$$

La primera condición evita que el robot avance cuando el objetivo se encuentra significativamente desalineado respecto al eje óptico de la cámara. La segunda introduce un comportamiento más conservador durante la recuperación de alineación, evitando movimientos oscilatorios y reduciendo la probabilidad de pérdida visual del objetivo.

Finalmente, cuando la distancia estimada alcanza la referencia deseada

$$\rho_k \leq \rho^*$$

el controlador detiene completamente el robot y el sistema transita a la siguiente fase de la misión.

3.5.4 Sampled Model Predictive Control

La optimización del controlador se implementa mediante un esquema de Sampled Model Predictive Control (Sampled MPC). A diferencia de un MPC convencional basado en solucionadores numéricos continuos, el enfoque adoptado discretiza explícitamente el espacio de acciones posibles y evalúa exhaustivamente el costo asociado a cada combinación de comandos candidatos.

El conjunto discreto de velocidades lineales candidatas se genera como

$$v_k \in \{v_1, v_2, \dots, v_n\}$$

mientras que las velocidades angulares candidatas pertenecen a

$$\omega_k \in \{\omega_1, \omega_2, \dots, \omega_m\}$$

En la implementación experimental se utilizaron 7 velocidades lineales discretas y 17 velocidades angulares discretas, generadas dentro de los límites físicos definidos para la plataforma.

Para cada par candidato (v_k, ω_k) , el controlador simula la evolución futura del estado utilizando el modelo predictivo descrito en las Ecuaciones (10) y (11) a lo largo de un horizonte de predicción de longitud N . Para cada combinación candidata de velocidades lineal y angular, el modelo predice cómo evolucionarán las variables visuales futuras bajo la aplicación sostenida de dicha acción.

El costo acumulado se calcula utilizando la función definida en la Ecuación (12), y finalmente se selecciona la acción que minimiza el costo total:

$$\mathbf{u}^* = \arg \min_{\mathbf{u}} J(\mathbf{x}, \mathbf{u}) \quad (17)$$

El horizonte de predicción utilizado fue $N = 10$, con un periodo de discretización de $\Delta t = 0.1$ s, lo cual corresponde a una ventana de predicción aproximada de un segundo hacia el futuro.

El controlador opera bajo un esquema de horizonte recedente (*receding horizon*). En cada iteración, el sistema obtiene una nueva estimación visual de ρ y α , ejecuta nuevamente la optimización sobre el conjunto discreto de acciones y aplica únicamente el mejor comando calculado para el instante actual. Este proceso se repite continuamente durante toda la aproximación visual.

3.5.5 Justificación de la Elección de MPC

La elección de Model Predictive Control (MPC) sobre controladores proporcionales clásicos responde a varias ventajas relevantes para la tarea de visual servoing considerada en este trabajo. Ya que el MPC permite incorporar explícitamente restricciones sobre las acciones de control. Esto resulta particularmente importante en plataformas diferenciales reales, donde las velocidades lineales y angulares se encuentran limitadas tanto por la dinámica física del robot como por consideraciones de estabilidad visual. Durante las pruebas experimentales se observó que comandos angulares excesivos podían provocar movimientos abruptos de la cámara, variaciones rápidas del bounding box y pérdida temporal del objetivo visual. La formulación predictiva permite restringir este comportamiento mediante penalizaciones y límites explícitos sobre las acciones de control.

Otro beneficio es que, el horizonte de predicción permite anticipar la evolución futura de las variables visuales antes de ejecutar una acción. Esto resulta especialmente útil debido al acoplamiento existente entre percepción y movimiento en sistemas de visual servoing. En este tipo de tareas, una acción de control inadecuada puede deteriorar la calidad de la percepción visual, lo que a su vez afecta negativamente las decisiones de control posteriores.

Adicionalmente, la formulación mediante una función de costo permite optimizar simultáneamente múltiples objetivos de desempeño, tales como:

- minimizar el error de distancia respecto al objetivo,
- mantener alineamiento visual estable,
- reducir el esfuerzo de control,
- suavizar las acciones enviadas al robot.

Esta flexibilidad resulta difícil de obtener mediante controladores proporcionales tradicionales, los cuales normalmente requieren ajustes heurísticos independientes para cada comportamiento deseado.

En este trabajo se optó específicamente por un enfoque de Sampled Model Predictive Control debido a la simplicidad computacional requerida por la plataforma y a la naturaleza de baja dimensionalidad del problema. A diferencia de un MPC convencional basado en solucionadores de optimización continua, el Sampled MPC implementado evalúa un conjunto discreto de acciones candidatas de velocidad lineal y angular, permitiendo realizar búsqueda exhaustiva en tiempo real sin necesidad de solucionadores numéricos complejos.

3.6 Controlador de Regreso por Odometría

Después de que el robot alcanza la distancia objetivo respecto a la caja seleccionada, el sistema considera finalizada la fase de recolección y simula la recolección del objeto. A partir de este punto, el robot deja de utilizar el controlador MPC y pasa a una etapa de decisión, en la cual espera una nueva instrucción del operador. El operador puede indicar que el robot regrese a la estación inicial mediante el comando `home`, o bien enviarlo hacia una coordenada específica mediante un comando de tipo `waypoint x y`.

Para el regreso a la estación de entrega, se define una pose fija de referencia:

$$P_{\text{home}} = (x_h, y_h, \theta_h) = (0, 0, 0) \quad (18)$$

Esta pose corresponde al origen del marco de odometría y representa el punto al que el robot debe regresar cuando recibe el comando **home**. Durante toda la ejecución, el sistema cuenta con la estimación actual de la pose del robot:

$$P = (x, y, \theta) \quad (19)$$

la cual es proporcionada por el nodo de odometría a través del tópico `/odom`.

El control de regreso se implementa mediante un controlador proporcional por odometría. En primer lugar, se calcula la diferencia entre la posición actual del robot y la posición objetivo:

$$e_x = x_h - x \quad (20)$$

$$e_y = y_h - y \quad (21)$$

A partir de estos errores se obtiene la distancia restante al punto *home*:

$$e_d = \sqrt{e_x^2 + e_y^2} \quad (22)$$

También se calcula el ángulo de dirección hacia el objetivo:

$$\theta_d = \tan^{-1} \left(\frac{e_y}{e_x} \right) \quad (23)$$

y el error angular entre la orientación actual del robot y la dirección deseada:

$$e_\theta = \text{wrap}(\theta_d - \theta) \quad (24)$$

donde la función $\text{wrap}(\cdot)$ normaliza el ángulo dentro del intervalo $[-\pi, \pi]$, evitando discontinuidades en la representación angular.

Mientras el robot no haya alcanzado la posición objetivo, las velocidades lineal y angular se calculan de forma proporcional a los errores de distancia y orientación:

$$v = k_d e_d \quad (25)$$

$$\omega = k_\theta e_\theta \quad (26)$$

Sin embargo, estos comandos se saturan para respetar los límites de operación definidos para la navegación por odometría:

$$0 \leq v \leq v_{\max}^{\text{ret}} \quad (27)$$

$$-\omega_{\max}^{\text{ret}} \leq \omega \leq \omega_{\max}^{\text{ret}} \quad (28)$$

En la implementación se utilizaron los siguientes valores:

$$v_{\max}^{\text{ret}} = 0.10 \text{ m/s} \quad (29)$$

$$\omega_{\max}^{\text{ret}} = 0.60 \text{ rad/s} \quad (30)$$

Además, para evitar que el robot avance mientras está demasiado desalineado respecto al punto objetivo, se aplica una condición de seguridad. Si el error angular supera los 25° , la velocidad lineal se anula y el robot únicamente gira hasta recuperar una orientación adecuada:

$$|e_\theta| > 25^\circ \Rightarrow v = 0 \quad (31)$$

El regreso a *home* se realiza en dos fases. En la primera, el robot se aproxima a la posición (x_h, y_h) hasta que la distancia residual es menor o igual a una tolerancia definida:

$$e_d \leq e_d^{tol} \quad (32)$$

donde:

$$e_d^{tol} = 0.08 \text{ m} \quad (33)$$

Una vez alcanzada la posición, comienza una segunda fase en la que se corrige la orientación final del robot para alinearla con la orientación de *home*, $\theta_h = 0$. El error de orientación final se define como:

$$e_{\theta_f} = \text{wrap}(\theta_h - \theta) \quad (34)$$

Durante esta fase, el robot no avanza y solo aplica velocidad angular:

$$v = 0 \quad (35)$$

$$\omega = k_f e_{\theta_f} \quad (36)$$

La misión de regreso termina cuando el error de orientación final se encuentra dentro de la tolerancia angular establecida:

$$|e_{\theta_f}| \leq e_\theta^{tol} \quad (37)$$

con:

$$e_\theta^{tol} = 8^\circ \quad (38)$$

Cuando se cumplen ambas condiciones, posición y orientación, el controlador detiene al robot y el sistema pasa al estado **FINISHED**.

Aunque esta sección se centra en el regreso a *home*, la implementación del controlador se desarrolló de manera general mediante la función `go_to_pose_controller`. Gracias a esto, la misma lógica de navegación por odometría puede reutilizarse para enviar al robot a un *waypoint* arbitrario (x_w, y_w) . En ese caso, el robot solo regula su posición y no realiza una corrección de orientación final, ya que el objetivo del *waypoint* es únicamente alcanzar una coordenada intermedia.

Siendo que el regreso a *home* o la navegación a *waypoints* se realiza con información de odometría, el desempeño de esta fase depende de la calidad de la estimación de pose y puede verse afectado por errores acumulados en los encoders, deslizamiento de las ruedas o pequeñas desviaciones durante el movimiento.

3.7 Máquina de Estados Finitos

La coordinación de las distintas fases del sistema se realiza mediante una máquina de estados finitos (FSM: Finite State Machine) que actúa como nivel supervisor del comportamiento del robot. La FSM determina, en cada instante, qué módulo de control se encuentra activo, qué condiciones deben cumplirse para realizar una transición, y cómo se interpreta la entrada del operador. La inclusión de una FSM resulta conveniente en nuestro sistema, porque se combinan múltiples regímenes de control sobre el mismo robot, ya que permite separar la lógica de alto nivel de la lógica de bajo nivel.

La FSM implementada consta de ocho estados, descritos en la Tabla 3. La transición entre estados ocurre cuando se satisface una condición específica del estado actual, ya sea derivada de una medición sensorial como distancia visual, error de pose, del transcurso de un intervalo temporal, o de un comando recibido del operador a través del nodo de teleoperación.

Table 3: Estados de la FSM y su función.

Estado	Función
WAIT_COMMAND	Espera la pulsación de una tecla (g, p o y) que indique el color objetivo. El robot permanece detenido.
GO_TO_VIEWPOINT	Avance opcional en línea recta hacia una zona de observación predefinida. En la configuración por defecto este estado se encuentra deshabilitado, de modo que el robot transita directamente al seguimiento visual.
TRACK_TARGET	Estado de aproximación visual: ejecuta el controlador MPC sobre el estado $[\rho, \alpha]^T$ hasta alcanzar la distancia objetivo ρ^* . Es el único estado en el que se utiliza retroalimentación visual en lazo cerrado.
COLLECTING	Pausa breve (simulación de recolección de la caja). El robot permanece detenido durante un intervalo fijo antes de transitar a la fase de espera de la siguiente acción.
WAIT_NEXT_ACTION	Estado de espera tras la recolección de la caja. El sistema permanece detenido a la espera de un comando del operador, que puede ser regresar a la estación de entrega (h) o navegar a un waypoint específico (w x y).
GO_TO_WAYPOINT	Navegación a un waypoint (x_d, y_d) especificado por el operador, utilizando el controlador proporcional sobre la pose estimada por odometría descrito en la Sección 3.6.
RETURN_HOME	Regreso a la estación de entrega $P_0 = (0, 0, 0)$ mediante el mismo controlador proporcional, incluyendo la fase final de corrección de orientación.
FINISHED	Estado terminal de la misión. El robot permanece detenido a la espera de un comando de reinicio del operador.

Las transiciones entre estados se resumen en la Tabla 4, donde se indica el estado origen, el

estado destino y la condición que dispara la transición.

Adicionalmente, la FSM admite comandos externos del operador con distintas funciones. Las teclas **c** y **q** funcionan en cualquier estado: la primera reinicia el sistema y lo lleva al estado **WAIT_COMMAND**, mientras que la segunda termina la ejecución del nodo. Por otro lado, los comandos de navegación posterior (**h** para regresar al origen y **w x y** para dirigirse a un waypoint) son válidos cuando la máquina de estados se encuentra en **WAIT_NEXT_ACTION**; si se reciben en otro estado, el sistema los registra en el log pero no actúa sobre ellos, evitando transiciones inconsistentes.

Table 4: Transiciones de la FSM.

Origen	Destino	Condición de transición
WAIT_COMMAND	TRACK_TARGET	Tecla g , p o y
TRACK_TARGET	COLLECTING	$\rho \leq \rho^*$
COLLECTING	WAIT_NEXT_ACTION	Transcurrido 1.0s tras alcanzar la caja
WAIT_NEXT_ACTION	RETURN_HOME	Comando h (home)
WAIT_NEXT_ACTION	GO_TO_WAYPOINT	Comando w x y (waypoint)
GO_TO_WAYPOINT	WAIT_NEXT_ACTION	Waypoint alcanzado ($e_d \leq e_d^{tol}$)
RETURN_HOME	FINISHED	Pose P_0 alcanzada con orientación correcta
FINISHED	WAIT_COMMAND	Tecla c

Además de las transiciones nominales, la FSM incorpora un mecanismo de seguridad ante pérdida temporal del objetivo visual. Durante el estado **TRACK_TARGET**, si la detección del objetivo falla en una imagen (por ejemplo, debido a oclusión parcial, condiciones de iluminación conflictivas o salida temporal del campo de visión), se incrementa un contador interno de frames perdidos. Si dicho contador supera un umbral de $N_{lost} = 10$ frames consecutivos sin detección válida, el sistema considera que el objetivo ha sido perdido y detiene el robot, manteniéndose a la espera de una nueva detección. Este mecanismo evita que el controlador continúe operando con información visual obsoleta o ausente, lo cual podría producir comportamientos inestables.

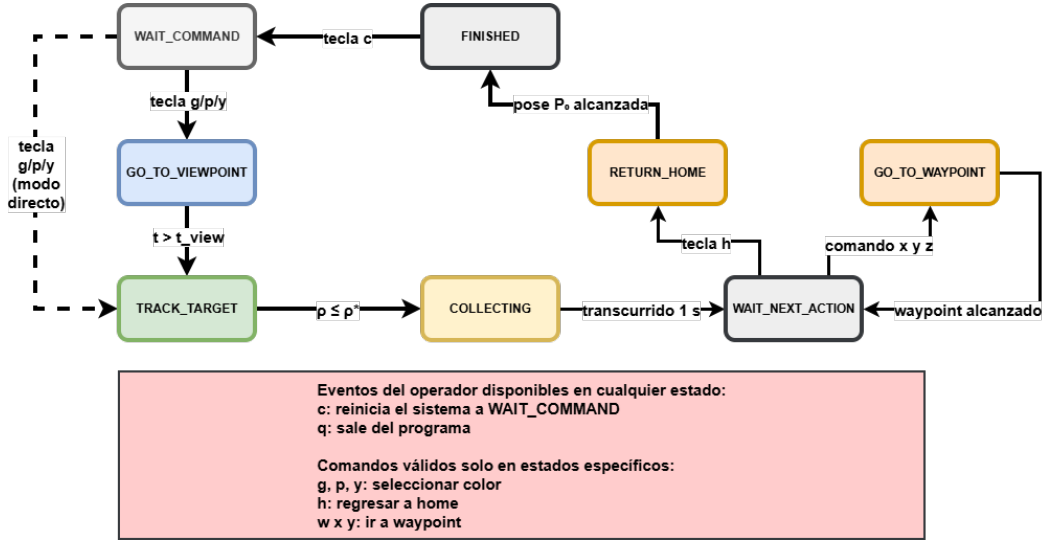


Figure 1: Diagrama de la máquina de estados finitos del sistema. Los estados sombreados corresponden a fases activas de control; los demás corresponden a fases de espera o terminales. La flecha punteada indica el flujo cuando el estado GO_TO_VIEWPOINT se encuentra deshabilitado, que es la configuración por defecto. Tras la recolección de la caja, el sistema pasa al estado WAIT_NEXT_ACTION, desde el cual el operador puede ordenar el regreso a la estación de entrega (RETURN_HOME) o la navegación a un waypoint arbitrario (GO_TO_WAYPOINT).

3.8 Pipeline Integrado

Las distintas etapas descritas en las subsecciones anteriores se integran en un pipeline que opera en lazo cerrado durante toda la misión. El sistema se compone de tres nodos ROS 2 independientes que se comunican exclusivamente mediante tópicos:

- El nodo `puzzlebot_odometry`, que integra las velocidades angulares de las ruedas y publica continuamente la pose estimada del robot.
- El nodo `teleop_color`, que captura las pulsaciones de tecla del operador desde la terminal y las traduce en mensajes de color objetivo.
- El nodo `puzzlebot_box_mpc_node`, que ejecuta el pipeline de visión clásica, el controlador MPC durante la fase de aproximación visual, el controlador proporcional por odometría utilizado tanto para el regreso a la estación de entrega como para la navegación a waypoints arbitrarios, y la máquina de estados que coordina todas las fases de la misión.

La Figura 2 resume la arquitectura completa del sistema, mostrando el flujo de información entre los componentes de percepción, modelado, control y actuación.

El ciclo de operación inicia cuando el operador presiona una tecla de color (g, p u y) en la terminal donde se ejecuta el nodo `teleop_color`. Este nodo publica un mensaje String con el nombre del color en el tópico `/box_color_command`. El nodo principal recibe este mensaje y dispara la transición de la máquina de estados desde WAIT_COMMAND hacia TRACK_TARGET.

Durante la fase de aproximación, cada imagen capturada por la cámara del Puzzlebot es procesada por el pipeline de visión (Sección 3.3), que extrae el centroide y el ancho aparente

del contorno correspondiente a la caja objetivo del color solicitado. Estas mediciones se transforman en el estado visual $\mathbf{x} = [\rho, \alpha]^T$ mediante el modelo de proyección pinhole descrito en la Sección 3.4.3. El controlador MPC recibe este estado, evalúa el conjunto discreto de acciones candidatas sobre el horizonte de predicción, y selecciona el par (v, ω) que minimiza la función de costo. Los comandos resultantes son publicados en el tópico `/cmd_vel` y ejecutados por el robot, cerrando así el lazo visual.

También, el nodo de odometría integra continuamente las velocidades angulares medidas en cada rueda mediante los encoders y publica la pose estimada en el tópico `/odom` a una frecuencia de 100 Hz. Esta información no es utilizada por el controlador durante la fase de aproximación visual, pero queda disponible de forma continua para la fase de regreso.

Cuando la distancia estimada ρ alcanza la referencia ρ^* , la FSM transita al estado `COLLECTING` y posteriormente a `WAIT_NEXT_ACTION`. En este estado, el control deja de operar sobre variables visuales y el robot permanece detenido a la espera de una nueva instrucción del operador. Desde esta fase, el operador puede ordenar el regreso a la estación de entrega $P_0 = (0, 0, 0)$ mediante el comando `h`, o solicitar la navegación a un waypoint con coordenadas arbitrarias (x_d, y_d) mediante el comando `w x y`. En ambos casos, el controlador proporcional sobre la pose estimada por odometría genera los comandos de velocidad correspondientes, descritos en la Sección 3.6, sin utilizar información visual durante la navegación. Una vez alcanzado un waypoint, el sistema regresa al estado `WAIT_NEXT_ACTION`, lo cual permite encadenar múltiples paradas dentro de la misma misión. Cuando finalmente el robot completa el regreso a P_0 , la FSM transita a `FINISHED` y el sistema queda disponible para recibir una nueva orden de color, permitiendo la operación continua a lo largo de múltiples entregas.

La arquitectura resultante combina percepción clásica, control óptimo visual y control de pose por odometría en un único sistema coherente, en el cual cada componente opera de forma desacoplada pero coordinada. La separación en nodos ROS 2 independientes facilita la depuración individual de cada módulo, el reemplazo futuro de cualquiera de ellos por una versión más sofisticada, y la posibilidad de reutilizar el pipeline sobre otras plataformas diferenciales con cambios mínimos en la configuración.

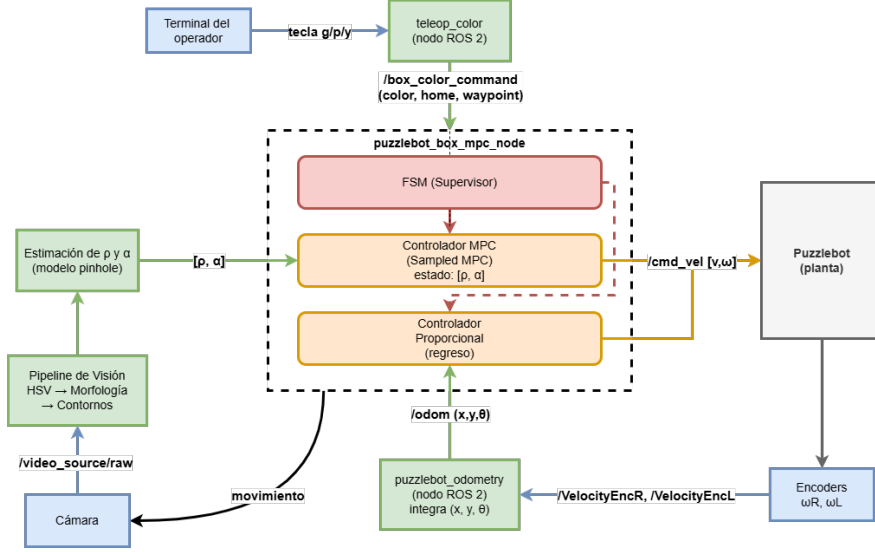


Figure 2: Arquitectura general del sistema. Los tres nodos ROS 2 se comunican mediante tópicos: `/box_color_command` transmite comandos del operador desde el nodo de teleoperación, incluyendo selección de color objetivo y solicitudes de navegación por odometría; `/video_source/raw` transmite las imágenes de la cámara; `/odom` transmite la pose estimada por odometría; y `/cmd_vel` transmite los comandos de velocidad al Puzzlebot. La FSM coordina las distintas fases de la misión, activando el controlador MPC durante la aproximación visual y el controlador proporcional durante la navegación posterior por odometría.

4 Resultados y Discusión

4.1 Configuración Experimental

Los experimentos se realizaron utilizando un robot diferencial Puzzlebot equipado con una Raspberry Pi Camera v2 montada frontalmente. Las pruebas se llevaron a cabo en un entorno interior con iluminación controlada y utilizando cajas de distintos colores como objetivos visuales. La distancia inicial entre el robot y el objetivo fue de aproximadamente 1m. Aunque el sistema fue diseñado para operar con tres colores objetivo (verde, rosa y amarillo), la prueba se realizó utilizando únicamente cuatro cajas: dos verdes y dos rosas. Ya que las cajas de color amarillo se parecían demasiado a las de color verde. Durante cada prueba, el robot debía:

- detectar visualmente la caja objetivo mas cercana
- aproximarse utilizando visual servoing basado en MPC
- navegar hacia un waypoint predefinido para simular la entrega del objeto
- regresar autónomamente a la posición inicial (HOME)

El controlador predictivo operó con un periodo de discretización de $\Delta t = 0.1$ s y un horizonte de predicción de $N = 10$.

4.2 Métricas Cuantitativas

Las principales métricas utilizadas para evaluar el desempeño del sistema fueron:

- error de distancia respecto a la referencia
- error angular durante la aproximación visual
- tiempo total de convergencia
- estabilidad de las señales de control
- porcentaje de detección visual durante la misión

4.3 Evolución Temporal del Sistema

Con el objetivo de analizar el comportamiento dinámico del sistema durante la ejecución completa de la misión, se registraron continuamente las variables internas del controlador y de la máquina de estados finitos. Las señales almacenadas incluyen las variables visuales utilizadas por el controlador predictivo, las acciones de control generadas, la pose odométrica estimada del robot, el costo asociado al MPC y el estado activo del sistema durante cada instante de tiempo.

A partir de estos datos se construyeron las gráficas temporales presentadas en esta sección, las cuales permiten evaluar el desempeño del controlador durante las distintas fases de operación, incluyendo:

- aproximación visual hacia la caja objetivo mediante MPC
- navegación hacia el waypoint de entrega
- regreso autónomo hacia la posición inicial

La Figura 3 muestra la trayectoria global del robot en el plano XY durante múltiples ciclos completos de misión. Cada trayectoria incluye las fases de aproximación visual hacia la caja objetivo, navegación hacia el waypoint de entrega y regreso autónomo hacia la posición inicial.

Durante la fase de visual servoing, el controlador MPC genera trayectorias suaves y progresivas hacia el objetivo visual. Una vez alcanzada la distancia deseada, el sistema cambia a navegación basada en odometría mediante un controlador proporcional, permitiendo el desplazamiento hacia el waypoint y posteriormente hacia la posición *home*.

La codificación temporal mediante color permite observar la ejecución secuencial de múltiples misiones autónomas. En todos los experimentos, el sistema fue capaz de completar exitosamente el ciclo completo de aproximación, entrega y retorno.

Asimismo, puede observarse una ligera acumulación de error espacial durante los regresos sucesivos, atribuida principalmente a errores odométricos acumulados y a las limitaciones inherentes de la navegación basada únicamente en encoders.

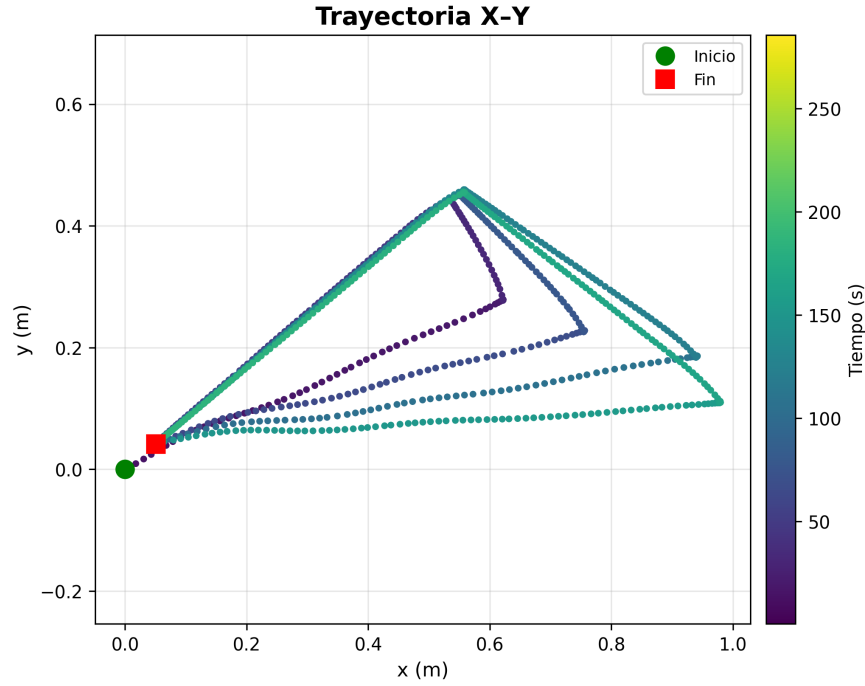


Figure 3: Trayectoria global del robot durante múltiples ciclos autónomos de aproximación visual, navegación hacia waypoint y regreso a la posición inicial.

La Figura 4 muestra la evolución temporal del error de distancia durante la prueba. El error se calcula con respecto a la distancia deseada de aproximación utilizada por el controlador predictivo.

$$e_{\rho}(t) = \rho(t) - \rho^*$$

Puede observarse que, durante cada fase de seguimiento visual, el controlador MPC reduce progresivamente el error de distancia hasta valores cercanos a cero, indicando convergencia estable hacia la referencia deseada. Las discontinuidades presentes en la señal no corresponden a fallas del controlador ni a pérdidas de percepción visual. Dichas interrupciones aparecen debido a que el robot únicamente utiliza las variables visuales ρ y α durante la fase activa de visual servoing. Una vez alcanzado el objetivo, el sistema cambia a modos de navegación basados en odometría para dirigirse hacia el waypoint de entrega y posteriormente regresar a la posición inicial. Durante estas etapas no existe un objetivo visual activo, por lo que el error de distancia deja de estar definido físicamente.

Asimismo, se observan algunos picos de error elevados durante ciertas transiciones de misión. Estos incrementos corresponden al inicio de nuevas tareas de seguimiento visual, donde el robot recibe un nuevo objetivo ubicado a mayor distancia respecto a la cámara.

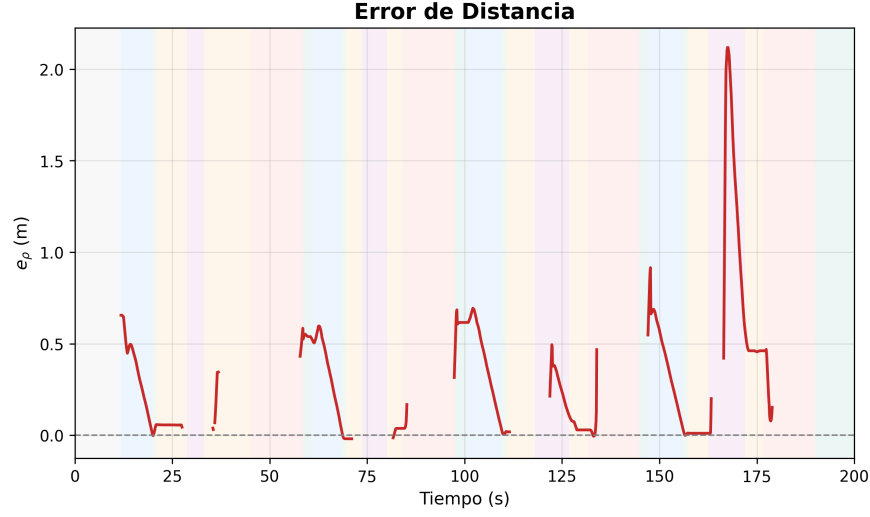


Figure 4: Evolución temporal del error de distancia durante las fases de visual servoing basadas en MPC.

La Figura 5 presenta la evolución temporal del error angular durante las distintas fases de visual servoing ejecutadas por el robot.

$$e_\alpha(t) = \alpha(t)$$

El error angular representa la desviación entre el eje óptico de la cámara y la dirección hacia la caja objetivo. Por lo tanto, valores cercanos a cero indican que el objeto permanece alineado con el centro de la imagen. Durante las fases activas de seguimiento visual, el controlador MPC mantiene el error angular acotado y realiza correcciones continuas de orientación para conservar el objetivo dentro del campo visual de la cámara.

Puede observarse que el sistema presenta variaciones angulares más abruptas que las observadas en el error de distancia. Este comportamiento es esperado debido a que pequeñas variaciones en la detección del bounding box producen cambios relativamente grandes en la estimación angular, especialmente cuando el objeto se encuentra cerca de los límites laterales de la imagen.

Asimismo, los cambios rápidos de signo en algunas regiones de la señal corresponden a maniobras de corrección ejecutadas por el robot para recentrar visualmente el objetivo durante la aproximación.

Las discontinuidades presentes en la gráfica corresponden nuevamente a las etapas de navegación no visual del sistema híbrido. Durante los movimientos hacia el waypoint de entrega y el regreso a la posición *home*, el robot deja de utilizar información visual activa y, por consiguiente, el error angular deja de estar definido.

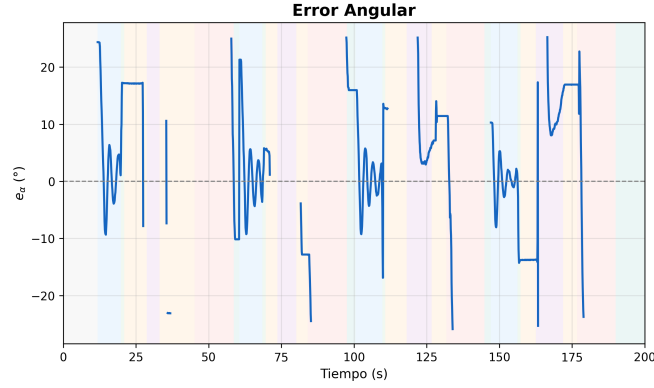


Figure 5: Evolución temporal del error angular durante las fases de seguimiento visual basadas en MPC.

La Figura 6 muestra la evolución temporal de las señales de control generadas por el sistema durante la ejecución de múltiples ciclos completos de misión.

La señal v_{cmd} corresponde a la velocidad lineal aplicada al robot, mientras que w_{cmd} representa la velocidad angular utilizada para mantener el alineamiento visual con el objetivo durante la fase de visual servoing.

Puede observarse que la velocidad lineal permanece acotada dentro de los límites definidos experimentalmente, manteniendo valores cercanos a 0.1 m/s durante las aproximaciones visuales. Por otro lado, la señal angular presenta variaciones más abruptas y frecuentes, especialmente durante las etapas iniciales de aproximación.

Durante las fases de navegación hacia waypoint y regreso a la posición *home*, las señales de control presentan comportamientos más simples y suaves, consistentes con la operación del controlador proporcional basado en odometría utilizado en dichas etapas.

Finalmente, puede observarse que ambas señales convergen a cero al final de cada misión, indicando que el robot alcanza exitosamente una condición estacionaria antes de esperar un nuevo objetivo.

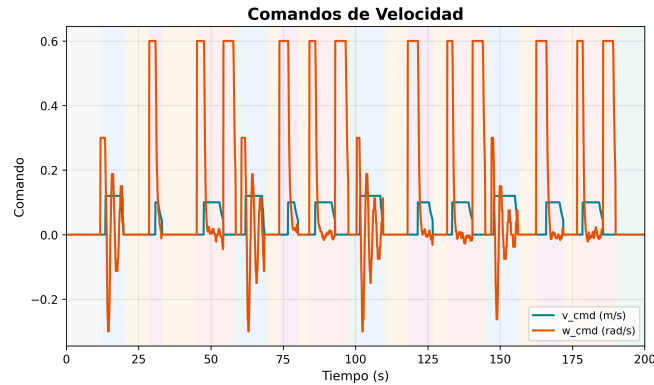


Figure 6: Evolución temporal de las velocidades lineal y angular generadas por el sistema de control.

La Figura 7 presenta la evolución temporal del valor de la función de costo utilizada por el controlador predictivo durante las distintas fases de visual servoing.

La función de costo integra simultáneamente términos asociados al error de distancia, error angular, esfuerzo de control y suavidad de las acciones de control. Por lo tanto, valores elevados de costo indican configuraciones del sistema alejadas de la referencia deseada o situaciones que requieren acciones de control más agresivas.

Puede observarse que, al inicio de cada nueva misión de seguimiento visual, el costo presenta valores elevados debido a que el robot se encuentra inicialmente lejos del objetivo y con errores de alineación significativos. Conforme avanza la aproximación visual, el valor del costo disminuye progresivamente hasta valores cercanos a cero. Este comportamiento evidencia que el controlador MPC logra reducir simultáneamente los errores visuales y las acciones de control requeridas para alcanzar la referencia deseada.

Finalmente, las discontinuidades presentes entre regiones de costo corresponden a los intervalos de navegación no visual del sistema, durante los cuales el controlador MPC deja de ejecutarse mientras el robot navega hacia el waypoint de entrega o regresa mediante odometría a la posición inicial.

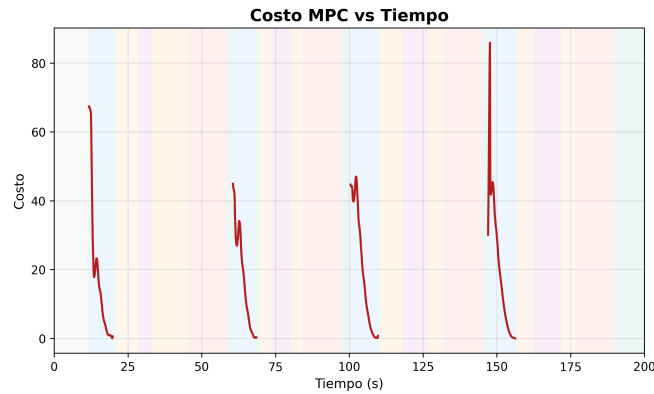


Figure 7: Evolución temporal de la función de costo del controlador MPC durante múltiples aproximaciones visuales.

La Figura 8 muestra la evolución temporal de la máquina de estados finitos (FSM) utilizada para coordinar las distintas etapas de la misión autónoma.

Puede observarse la secuencia completa de operación del sistema, iniciando en `WAIT_COMMAND`, seguida de la fase de seguimiento visual mediante `TRACK_TARGET`, la simulación de recolección en `COLLECTING`, la navegación hacia el punto de entrega en `GO_TO_WAYPOINT` y finalmente el regreso autónomo mediante `RETURN_HOME`.

Asimismo, el estado `WAIT_NEXT_ACTION` se utiliza como transición temporal entre algunas etapas de la misión, mientras que el estado `FINISHED` indica la conclusión exitosa de cada ciclo.

La figura evidencia que el sistema fue capaz de ejecutar múltiples misiones completas de manera consistente, validando el funcionamiento de la arquitectura híbrida propuesta.

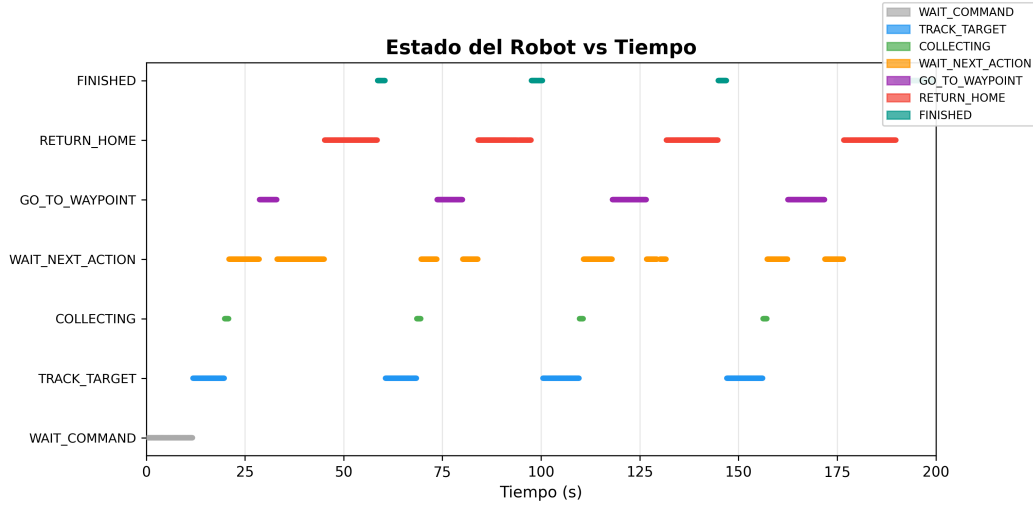


Figure 8: Evolución temporal de los estados de la FSM durante múltiples ciclos de misión.

4.4 Modos de Falla Observados

Durante las pruebas experimentales se observaron algunas limitaciones relacionadas principalmente con la percepción visual y la acumulación de error odométrico.

En ciertas situaciones, movimientos angulares rápidos provocaron variaciones abruptas en el bounding box detectado, generando pérdidas temporales del objetivo visual. Asimismo, durante trayectorias largas de regreso se observó acumulación gradual de error odométrico, afectando ligeramente la precisión final del retorno a la pose inicial.

5 Conclusiones y Trabajo Futuro

5.1 Conclusiones

En este trabajo se desarrolló un sistema de visual servoing basado en imagen para que un robot móvil diferencial pudiera identificar y aproximarse a cajas según el color solicitado por el operador. El sistema integró visión clásica, estimación de variables visuales, control predictivo, navegación por odometría y una máquina de estados para coordinar toda la misión.

A partir de la imagen capturada por la cámara, se estimaron la distancia aproximada a la caja, ρ , y el error angular, α . Estas dos variables permitieron representar de forma simple la posición relativa del objetivo y fueron utilizadas directamente por el controlador MPC para generar los comandos de velocidad del robot.

Los resultados experimentales mostraron que el sistema fue capaz de detectar el objetivo, aproximarse a él de forma estable y reducir progresivamente el error de distancia. Además, el controlador mantuvo el error angular dentro de rangos adecuados y generó movimientos más suaves durante la aproximación.

Después de alcanzar la caja, el robot pudo continuar la misión mediante navegación por odometría, ya sea hacia un waypoint de entrega o de regreso a la posición inicial. Esto permitió completar varios ciclos de operación dentro de una misma prueba.

En general, el proyecto demuestra que es posible combinar visión clásica y control predictivo en un robot físico para resolver una tarea de navegación guiada visualmente en tiempo real. El sistema desarrollado cumplió con el objetivo principal de transformar información visual en acciones de control que permitieran al robot acercarse de manera autónoma a un objetivo seleccionado.

5.2 Limitaciones Reconocidas

A pesar de que el sistema logró completar la tarea propuesta, se identificaron algunas limitaciones importantes. La primera se relaciona con la percepción visual, ya que la segmentación por color depende de condiciones de iluminación relativamente estables. Cambios bruscos de luz, reflejos o sombras pueden afectar la detección del contorno y, por lo tanto, la estimación de las variables visuales.

También se observó que movimientos angulares rápidos del robot pueden provocar variaciones abruptas en el bounding box detectado. Esto genera fluctuaciones en la estimación de ρ y α , e incluso pérdidas temporales del objetivo visual en algunos instantes de la prueba.

Otra limitación relevante corresponde a la navegación basada únicamente en odometría. Durante trayectorias largas hacia waypoints o durante el regreso a la posición *home*, se presentó acumulación gradual de error debido a pequeñas imprecisiones de los encoders y posibles deslizamientos de las ruedas. Aunque este error no impidió completar las misiones evaluadas, sí afectó ligeramente la precisión final del retorno al origen.

Finalmente, la recolección de la caja fue representada de manera simulada mediante la llegada a una distancia visual objetivo, por lo que el sistema aún no incorpora un mecanismo físico de manipulación o arrastre del objeto.

5.3 Trabajo Futuro

Como trabajo futuro, una primera mejora consistiría en aumentar la robustez del pipeline de percepción frente a variaciones de iluminación. Esto podría lograrse mediante una calibración dinámica de umbrales HSV o normalización de color.

También sería conveniente mejorar la estabilidad de la estimación de las variables visuales durante movimientos angulares del robot. Una posible línea de trabajo sería incorporar filtros temporales más robustos o mecanismos de seguimiento del objetivo que reduzcan las fluctuaciones en el bounding box y eviten pérdidas breves de detección.

Para la navegación posterior a la recolección, se propone complementar la odometría con alguna fuente adicional de corrección, como marcadores visuales. Esto permitiría reducir la acumulación de error y mejorar la precisión del regreso a la estación inicial y del seguimiento de waypoints.

Otra extensión natural del proyecto sería incorporar un mecanismo físico de recolección o empuje de las cajas y se convierta en una operación de entrega más completa.

References

- [1] F. Chaumette and S. Hutchinson, “Visual servo control. Part I: Basic approaches,” *IEEE Robotics & Automation Magazine*, vol. 13, no. 4, pp. 82–90, Dec. 2006, doi: 10.1109/MRA.2006.250573.
- [2] F. Chaumette and S. Hutchinson, “Visual servo control. Part II: Advanced approaches,” *IEEE Robotics & Automation Magazine*, vol. 14, no. 1, pp. 109–118, Mar. 2007, doi: 10.1109/MRA.2007.339609.

- [3] H. Bai, J. Gao, X. Sun, and W. Yan, “Model predictive visual trajectory-tracking control of wheeled mobile robots,” in *Proc. 2019 IEEE 28th International Symposium on Industrial Electronics (ISIE)*, Vancouver, BC, Canada, 2019, pp. 569–574, doi: 10.1109/ISIE.2019.8781161.
- [4] J. Deng and Z. Li, “Model predictive control for visual servo steering of nonholonomic mobile robots,” in *Proc. 11th World Congress on Intelligent Control and Automation*, Shenyang, China, 2014, pp. 347–352, doi: 10.1109/WCICA.2014.7052738.
- [5] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, May 2022, doi: 10.1126/scirobotics.abm6074.
- [6] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, 2000.
- [7] Manchester Robotics, “Puzzlebot educational robotics platform,” 2024. [Online]. Available: <https://www.manchester-robotics.com>

Anexos

- Repositorio en Github: https://github.com/Jose05M/puzzlebot_box_mpc/tree/main
- Link del video en drive donde se demuestra el funcionamiento del proyecto: https://drive.google.com/file/d/1wcS1_m62bl7JUfi-15h4jyT4J4-Xzl2c/view?usp=sharing